

**LAPORAN TUGAS SISTEM OPERASI
SISTEM BOOKING HOTEL(AKHIRNYA NIM GANJIL)**



Dosen Pengampu Matakuliah :

Triawan Adi Cahyanto, M.K

Disusun Oleh :

Muhammad Gufron (2410651053)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

TAHUN AKADEMIK 2024/2025

DAFTAR ISI

Daftar Isi	
BAB I	4
PENDAHULUAN	4
BAB II	5
LANDASAN TIORI	5
2.1. Deskripsi Masalah	5
2.2. Requirements	5
BAB III	6
PEMBAHASAN	6
3.1. Analisis Masalah	6
3.2. Implementasi	6
3.3. Testing & Output	11
3.4. Analisis Hasil	12
BAB IV	13
PENUTUP	13
4.1. Kesimpulan	13

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat dan hidayah-Nya sehingga kami dapat menyelesaikan tugas makalah yang berjudul “Memori Dalam Komputer” ini tepat waktu.

Adapun tujuan dari penulisan makalah ini adalah untuk memenuhi tugas dosen pada mata kuliah Dasar-Dasar Sistem. Selain itu, makalah ini juga bertujuan untuk menambah wawasan tentang Kerja Sama Kelompok bagi para pembaca dan juga bagi penulis.

Kami mengucapkan terima kasih kepada Triawan Adi Cahyanto, M.K

selaku dosen mata kuliah Pengembangan Diri yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan bidang studi yang kami tekuni.

Kami juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga kami dapat menyelesaikan makalah ini. Kami menyadari, makalah yang kami tulis ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun akan kami nantikan demi kesempurnaan makalah ini.

BAB I

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi informasi telah mendorong peningkatan efisiensi dalam berbagai sektor, termasuk industri perhotelan. Sistem booking hotel yang andal dan efisien menjadi kebutuhan utama untuk mengelola permintaan pemesanan kamar yang tinggi, terutama pada periode tertentu seperti musim liburan atau acara khusus. Dalam konteks ini, sistem operasi memegang peran penting dalam mengelola sumber daya secara bersamaan, seperti akses ke kamar hotel oleh banyak pelanggan dalam waktu yang hampir bersamaan.

Simulasi sistem booking hotel ini dirancang untuk mengimplementasikan konsep multithreading dan sinkronisasi proses dalam sistem operasi. Dengan menggunakan mekanisme semaphore dan mutex, sistem ini dapat mengatur akses terhadap kamar yang terbatas serta mencegah terjadinya race condition yang dapat menyebabkan inkonsistensi data. Melalui simulasi ini, diharapkan dapat dipahami bagaimana sistem operasi mengelola sumber daya bersama secara aman dan efisien.

1.2. Rumusan Masalah

1. Bagaimana mengimplementasikan sistem booking hotel yang melibatkan banyak pelanggan (thread) yang mengakses kamar secara bersamaan ?
2. Bagaimana mengatur sinkronisasi akses ke kamar hotel yang terbatas menggunakan semaphore ?
3. bagaimana menjaga konsistensi output saat beberapa thread mencoba menulis informasi secara bersamaan ?

1.3. Tujuan Penulisan

1. Memahami konsep multithreading dan sinkronisasi proses dalam sistem operasi.
2. Mengimplementasikan semaphore dan mutex untuk mengelola akses ke sumber daya bersama.
3. Membuat simulasi sistem booking hotel yang dapat menangani banyak pelanggan secara bersamaan dengan output yang rapi dan informatif.

BAB II

LANDASAN TIORI

2.1. Deskripsi Masalah

Anda diminta membuat simulasi sistem booking hotel dengan kondisi :

- Hotel memiliki 5 kamar yang bisa diboeking
- Ada 10 customer yang mencoba booking secara bersamaan (10 threads)
 - Setiap customer akan:
 1. Mencoba booking kamar (butuh waktu random 1-3 detik)
 2. Jika berhasil, "menginap" selama 5 detik
 3. Check-out dan melepaskan kamar
 4. Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi
 - Customer maksimal mencoba 3 kali, jika gagal terus maka menyerah

2.2. Requirements

1. Gunakan Semaphore untuk mengatur jumlah kamar tersedia
2. Gunakan Mutex untuk melindungi operasi print (agar output tidak berantakan)
3. Tampilkan informasi:
 - Customer ID berapa yang berhasil booking
 - Kamar mana yang di tempati
 - Berapa kamar yang masih tersedia
 - Customer yang gagal booking
4. Output harus rapi dan mudah dibaca (gunakan emoji atau format yang jelas)

BAB III

PEMBAHASAN

3.1. Analisis Masalah

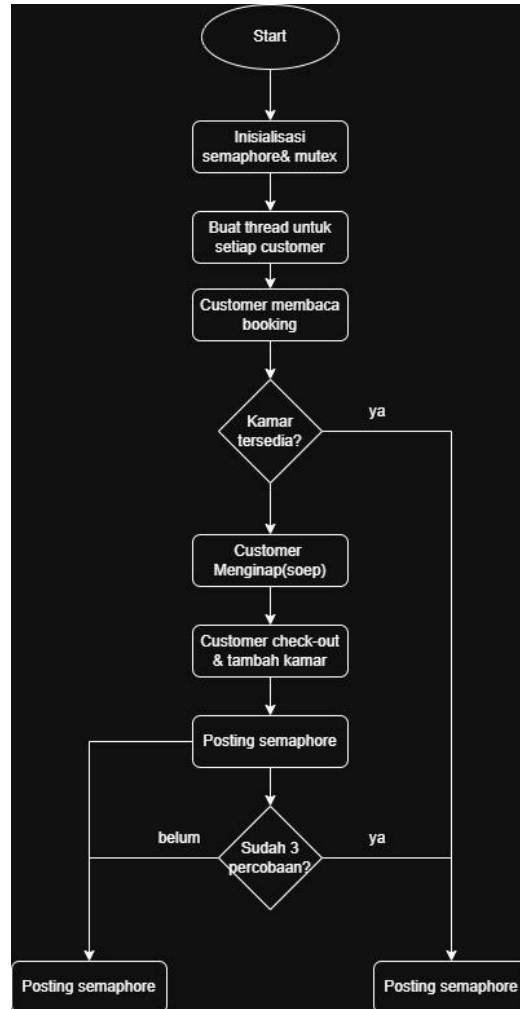
Program di atas mensimulasikan sistem booking kamar hotel dengan multithreading menggunakan pthread dan semaphore untuk mengelola sinkronisasi antar thread. Terdapat 10 pelanggan (thread) yang bersaing untuk memesan 5 kamar yang tersedia. Masalah utama yang dihadapi adalah sinkronisasi akses terhadap sumber daya terbatas, yaitu jumlah kamar. Tanpa mekanisme sinkronisasi, dua atau lebih thread dapat mengakses dan mengubah data kamar secara bersamaan sehingga menyebabkan inkonsistensi. Untuk mengatasi hal ini, digunakan semaphore sebagai kontrol jumlah kamar yang dapat diakses, dan mutex untuk mengamankan proses print agar output tidak tumpang tindih. Desain solusi ini memastikan eksekusi program tetap teratur, aman, dan realistis.

3.2. Implementasi

1. Penjelasan algoritma

Program ini mensimulasikan proses sinkronisasi pemesanan kamar hotel oleh beberapa pelanggan secara bersamaan menggunakan *multithreading* dengan pthread. Setiap pelanggan dijalankan sebagai thread terpisah yang berusaha memesan kamar terbatas secara bersamaan. Sinkronisasi dilakukan menggunakan semaphore untuk mengontrol jumlah kamar yang tersedia, dan mutex untuk mengatur akses ke output agar tidak terjadi *race condition*. Jika semua kamar penuh, pelanggan akan mencoba lagi hingga tiga kali sebelum menyerah. Setelah selesai menginap, pelanggan melakukan *check-out*, dan kamar tersebut dikembalikan ke daftar kamar tersedia.

2. Flowchart/diagram



3. Potongan kode penting dengan penjelasan

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

#define JUMLAH_KAMAR 5
#define JUMLAH_CUSTOMER 10

sem_t kamarTersedia;
pthread_mutex_t mutexPrint;
int kamarTersisa = JUMLAH_KAMAR;
```

```

void* customer(void* arg) {
    int id = *((int*) arg);
    int percobaan = 0;

    while (percobaan < 3) {
        percobaan++;
        sleep(rand() % 3 + 1); // waktu mencoba booking (1–3 detik)

        pthread_mutex_lock(&mutexPrint);
        printf("🕒 Customer-%d mencoba booking (percobaan ke-%d)\n", id, percobaan);
        pthread_mutex_unlock(&mutexPrint);

        // coba booking kamar (semaphore)
        if (sem_trywait(&kamarTersedia) == 0) {
            pthread_mutex_lock(&mutexPrint);
            kamarTersisa--;
            printf("✅ Customer-%d berhasil booking! (%d kamar tersisa)\n", id,
kamarTersisa);
            pthread_mutex_unlock(&mutexPrint);

            sleep(5); // menginap selama 5 detik

            pthread_mutex_lock(&mutexPrint);
            kamarTersisa++;
            printf("❏ Customer-%d check-out. (%d kamar tersisa)\n", id, kamarTersisa);
            pthread_mutex_unlock(&mutexPrint);

            sem_post(&kamarTersedia); // lepaskan kamar
            break; // selesai booking
        } else {
            pthread_mutex_lock(&mutexPrint);
            printf("❌ Customer-%d gagal booking (kamar penuh)\n", id);
            pthread_mutex_unlock(&mutexPrint);
            sleep(2); // tunggu 2 detik lalu coba lagi
        }
    }

    if (percobaan == 3) {
        pthread_mutex_lock(&mutexPrint);
        printf("Customer-%d menyerah (tidak dapat kamar)\n", id);
        pthread_mutex_unlock(&mutexPrint);
    }

    return NULL;
}

```



```

int main() {
    pthread_t threads[JUMLAH_CUSTOMER];
    int id[JUMLAH_CUSTOMER];
    srand(time(NULL));

    sem_init(&kamarTersedia, 0, JUMLAH_KAMAR);
    pthread_mutex_init(&mutexPrint, NULL);

    printf("=== SISTEM BOOKING HOTEL ===\n");
    printf("Total kamar: %d\n\n", JUMLAH_KAMAR);

    // buat thread untuk tiap customer
    for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
        id[i] = i + 1;
        pthread_create(&threads[i], NULL, customer, &id[i]);
    }

    // tunggu semua thread selesai
    for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
        pthread_join(threads[i], NULL);
    }

    sem_destroy(&kamarTersedia);
    pthread_mutex_destroy(&mutexPrint);

    printf("\n=== Simulasi selesai ===\n");
    return 0;
}

```

```

if (sem_trywait(&kamarTersedia) == 0) {
    pthread_mutex_lock(&mutexPrint);
    kamarTersisa--;
    printf("✅ Customer-%d berhasil booking! (%d kamar tersisa)\n", id,
kamarTersisa);
    pthread_mutex_unlock(&mutexPrint);
}

```

Kode ini memastikan hanya pelanggan yang berhasil memperoleh izin semaphore dapat mengurangi jumlah kamar secara sinkron dan aman.

```
sem_init(&kamarTersedia, 0, JUMLAH_KAMAR);  
pthread_mutex_init(&mutexPrint, NULL);
```

Inisialisasi semaphore dengan jumlah kamar awal dan mutex untuk mencegah konflik saat beberapa thread menulis ke konsol

```
sem_post(&kamarTersedia);
```

Digunakan saat pelanggan check-out untuk melepaskan kamar, menambah kembali satu izin semaphore agar bisa digunakan pelanggan lain.

4. Mengapa memilih mekanisme tertentu

Pemilihan semaphore pada program ini didasarkan pada kebutuhan untuk membatasi jumlah sumber daya bersama, yaitu kamar hotel, yang dapat diakses secara bersamaan oleh banyak thread. Semaphore efektif mengontrol berapa banyak thread yang dapat "masuk" pada saat tertentu, sehingga mencegah overbooking. Sementara itu, penggunaan mutex berfungsi untuk menjaga konsistensi tampilan output dan mencegah terjadinya race condition ketika beberapa thread mencoba menulis ke layar secara bersamaan.

3.3. Testing & Output

```
gufre@MSI:~/Praktikum5/sinkronisasi/percobaan$ nano booking_hotel.c
gufre@MSI:~/Praktikum5/sinkronisasi/percobaan$ gcc booking_hotel.c -o booking_hotel -pthread
gufre@MSI:~/Praktikum5/sinkronisasi/percobaan$ ./booking_hotel
=== SISTEM BOOKING HOTEL ===
Total kamar: 5

❶ Customer-1 mencoba booking (percobaan ke-1)
✅ Customer-1 berhasil booking! (4 kamar tersisa)
❶ Customer-7 mencoba booking (percobaan ke-1)
✅ Customer-7 berhasil booking! (3 kamar tersisa)
❶ Customer-5 mencoba booking (percobaan ke-1)
✅ Customer-5 berhasil booking! (2 kamar tersisa)
❶ Customer-4 mencoba booking (percobaan ke-1)
✅ Customer-4 berhasil booking! (1 kamar tersisa)
❶ Customer-10 mencoba booking (percobaan ke-1)
✅ Customer-10 berhasil booking! (0 kamar tersisa)
❶ Customer-3 mencoba booking (percobaan ke-1)
❌ Customer-3 gagal booking (kamar penuh)
❶ Customer-8 mencoba booking (percobaan ke-1)
❌ Customer-8 gagal booking (kamar penuh)
❶ Customer-9 mencoba booking (percobaan ke-1)
❌ Customer-9 gagal booking (kamar penuh)
❶ Customer-6 mencoba booking (percobaan ke-1)
❌ Customer-6 gagal booking (kamar penuh)
❶ Customer-2 mencoba booking (percobaan ke-1)
❌ Customer-2 gagal booking (kamar penuh)
❶ Customer-8 mencoba booking (percobaan ke-2)
❌ Customer-8 gagal booking (kamar penuh)
❶ Customer-1 check-out. (1 kamar tersisa)
❶ Customer-7 check-out. (2 kamar tersisa)
❶ Customer-3 mencoba booking (percobaan ke-2)
✅ Customer-3 berhasil booking! (1 kamar tersisa)
❶ Customer-5 check-out. (2 kamar tersisa)
❶ Customer-4 check-out. (3 kamar tersisa)
❶ Customer-6 mencoba booking (percobaan ke-2)
✅ Customer-6 berhasil booking! (2 kamar tersisa)
❶ Customer-10 check-out. (3 kamar tersisa)
❶ Customer-2 mencoba booking (percobaan ke-2)
✅ Customer-2 berhasil booking! (2 kamar tersisa)
❶ Customer-9 mencoba booking (percobaan ke-2)
✅ Customer-9 berhasil booking! (1 kamar tersisa)
❶ Customer-8 mencoba booking (percobaan ke-3)
✅ Customer-8 berhasil booking! (0 kamar tersisa)
❶ Customer-3 check-out. (1 kamar tersisa)
❶ Customer-6 check-out. (2 kamar tersisa)
❶ Customer-2 check-out. (3 kamar tersisa)
❶ Customer-9 check-out. (4 kamar tersisa)
❶ Customer-8 check-out. (5 kamar tersisa)
❶ Customer-8 menyerah (tidak dapat kamar)
```

Hasil pengujian program Booking Hotel menunjukkan bahwa sistem sinkronisasi antar-thread berjalan dengan baik menggunakan semaphore dan mutex. Pada awalnya terdapat lima kamar tersedia, kemudian setiap thread customer mencoba melakukan pemesanan. Jika kamar tersedia, proses booking berhasil dan jumlah kamar tersisa berkurang; sebaliknya, jika penuh, thread menunggu atau gagal melakukan booking. Setelah beberapa waktu, beberapa customer melakukan check-out sehingga kamar kembali tersedia untuk thread lain yang masih menunggu.

3.4. Analisis Hasil

Pada percobaan program simulasi booking hotel ini, hasil eksekusi menunjukkan bahwa sistem berjalan sesuai ekspektasi. Setiap customer thread berusaha melakukan pemesanan kamar secara paralel, dengan mekanisme semaphore memastikan jumlah kamar tidak melebihi kapasitas yang ditentukan. Saat kamar penuh, pelanggan menunggu dan mencoba kembali hingga tiga kali sebelum menyerah. Tidak ditemukan race condition karena penggunaan mutex pada bagian print menjaga keluaran tetap sinkron. Performa program cukup stabil meskipun terdapat delay acak, menunjukkan manajemen thread yang efisien. Dari simulasi ini, diperoleh pemahaman bahwa sinkronisasi antar-thread sangat penting untuk mencegah konflik sumber daya bersama.

BAB IV

PENUTUP

4.1. Kesimpulan

Berdasarkan implementasi dari simulasi sistem booking hotel ini menunjukkan bahwa penerapan multithreading dengan bantuan semaphore dan mutex mampu mengatur akses terhadap sumber daya terbatas secara efektif. Melalui program ini, dipelajari pentingnya sinkronisasi dalam mencegah race condition serta menjaga kestabilan eksekusi antar thread. Tantangan yang dihadapi adalah mengatur koordinasi antar thread agar tidak terjadi konflik dan memastikan output tetap rapi meskipun proses berjalan bersamaan. Untuk pengembangan selanjutnya, program dapat ditingkatkan dengan menambahkan sistem antrian, antarmuka pengguna interaktif, serta integrasi dengan basis data agar lebih realistis dan aplikatif